

Week 2 Day 3: 컨테이너 실행 옵션과 런타임 운영

Overview

Day 3는 Day 2에서 만든 image를 어떤 조건으로 실행할지 다룬다. Dockerfile은 image를 만드는 기준이고, `docker run` 옵션은 같은 image를 어떤 network, port, environment, volume 조건으로 실행할지 정하는 기준이다.

오늘의 핵심 질문은 다음과 같다.

같은 image라도 실행 옵션이 달라지면 container의 접근 경로, 설정, 데이터 보존, 장애 증거가 어떻게 달라지는가?

Day 3는 설명보다 실행 비중이 높다. 각 교시는 명령을 실행하고, `docker ps`, `curl`, `docker logs`, `docker inspect`, `docker exec` 로 evidence를 남긴다. 실행에 성공한 결과뿐 아니라 실패한 결과도 기록한다. 운영에서는 정상 명령보다 실패 상황을 얼마나 빠르게 분류하는지가 더 중요하다.

Learning Goals

- host port와 container port의 차이를 설명한다.
- `-p host:container` port binding을 실행하고 HTTP evidence를 확보한다.
- Docker bridge network와 container name/DNS 기반 접근을 설명한다.
- `-e` environment variable이 image가 아니라 runtime configuration임을 설명한다.
- bind mount와 named volume을 runtime storage 관점에서 구분한다.
- PostgreSQL container를 environment, network, volume과 함께 실행한다.
- `docker logs`, `docker exec`, `docker inspect` 로 장애 원인을 분류한다.
- README에 실행 조건과 cleanup 기준을 handoff 형식으로 기록한다.

Lesson Index

- 1교시: port binding과 localhost 접근 - `-p 18083:80`, HTTP 200
- 2교시: Docker bridge network와 container DNS - custom network, service name
- 3교시: environment variable - `-e`, runtime config, secret 주의
- 4교시: bind mount로 runtime file 제공 - host disk와 container path
- 5교시: named volume과 데이터 보존 - volume lifecycle, inspect
- 6교시: PostgreSQL container 실행 - env, network, volume, readiness
- 7교시: logs/exec/inspect 기반 장애 분석 - missing env, wrong port, stale volume
- 8교시: Day 3 README handoff와 cleanup audit - 실행 조건 문서화

Practice Files And Assets

자료	용도
labs/runtime-site/html/index.html	nginx bind mount HTTP 실습
labs/env-report/report.sh	environment variable 출력 실습
labs/env-report/README.md	env 실습 명령과 기대 결과
labs/postgres/README.md	PostgreSQL network/volume 실습
hands-on-lab.md	Day 3 전체 실행 흐름
assets/lesson-01-port-binding-localhost.png	port binding 다이어그램
assets/lesson-02-bridge-network-dns.png	bridge network/DNS 다이어그램
assets/lesson-03-runtime-env-vars.png	environment variable 다이어그램
assets/lesson-04-bind-mount-runtime-files.png	bind mount runtime file 흐름

assets/lesson-05-named-volume-lifecycle.png	named volume lifecycle
assets/lesson-06-postgres-runtime-contract.png	PostgreSQL runtime contract
assets/lesson-07-runtime-troubleshooting.png	runtime troubleshooting
assets/lesson-08-runtime-handoff-cleanup.png	README handoff와 cleanup

Linux Preflight Test Evidence

Day 3 명령은 Linux Docker 환경에서 사전 테스트했다.

항목	결과
Test OS	Ubuntu 24.04.3 LTS, Linux 6.6.87.2-microsoft-standard-WSL2, linux/amd64
Docker images	nginx:1.27-alpine 48.2MB, alpine:3.20 7.81MB, postgres:16-alpine 276MB
Web run	docker run -d --name paperclip-day3-web -p 18083:80 -v ...:ro nginx:1.27-alpine
Port evidence	0.0.0.0:18083->80/tcp, [::]:18083->80/tcp
HTTP evidence	HTTP/1.1 200 OK, Server: nginx/1.27.5, Content-Length: 1369
Bind mount evidence	source week2/day3/labs/runtime-site/html, destination /usr/share/nginx/html, RW:false
Env evidence	APP_ENV=practice, APP_PORT=8080, FEATURE_FLAG=on, DB_HOST=postgres, DB_PORT=5432
Network create	paperclip-day3-net 생성 성공
Volume create	paperclip-day3-pgdata 생성 성공
PostgreSQL run	paperclip-day3-postgres 실행, network paperclip-day3-net, volume mount /var/lib/postgresql/data
Readiness	pg_isready -U postgres -> /var/run/postgresql:5432 - accepting connections
SQL evidence	select current_database(); -> paperclip
DNS evidence	same network container에서 paperclip-day3-postgres:5432 - accepting connections
Missing env failure	POSTGRES_PASSWORD 누락 시 superuser password required error 확인

Required Evidence

Evidence	제출 기준
Port evidence	docker ps의 PORTS, curl -I HTTP status
Network evidence	custom network 이름, 연결된 container 이름, DNS 확인
Env evidence	-e로 주입한 key/value 출력
Bind mount evidence	source/destination/mode, RW:false 또는 :ro 확인
Volume evidence	named volume name, mount destination, container 삭제 후 보존 여부
DB evidence	pg_isready, SQL 결과, logs 핵심 줄
Failure evidence	wrong port, missing env, stale volume 중 하나의 RCA
Cleanup evidence	container, network, volume 정리 결과

End-Of-Day Checklist

- ☐ `-p host:container` 에서 host port와 container port를 구분했다.
- ☐ browser 또는 `curl` 로 `localhost:18083` HTTP 200을 확인했다.
- ☐ `docker inspect` 로 bind mount mode를 확인했다.
- ☐ `-e` environment variable을 container runtime에 주입했다.
- ☐ custom bridge network를 만들고 container DNS를 확인했다.
- ☐ PostgreSQL container를 named volume과 함께 실행했다.
- ☐ missing env 실패 로그를 RCA 형식으로 기록했다.
- ☐ cleanup audit을 수행했다.

Next Connection

Day 4는 Compose로 넘어간다. Day 3에서 길게 입력한 `docker run --network , -e , -v , -p` 옵션은 Day 4에서 `compose.yaml` 의 `networks` , `environment` , `volumes` , `ports` 로 옮겨진다.